Master's Thesis

Improvement of Simulation Performance

How Can Numerical Frameworks Be Improved by Algorithmic Optimizations? – A Case Study with ISSM

Lauritz Henrik Timm
Mat. 1190464

26.01.2026

Advisor:
Prof. Dr. Thomas Slawig Algorithmic Optimal Control

Examiner:
Prof. Dr. Steffen Börm Scientific Computing

Department of Computer Science
Faculty of Engineering
Kiel University

Kiel University
Christian-Albrechts-Universität zu Kiel

Declaration

I confirm, that the master's thesis "Improvement of Simulation Performance: How Can Numerical Frameworks Be Improved by Algorithmic Optimizations? – A Case Study with ISSM" is the result of my own work. No other person's work has been used without acknowledgment in the main text of this thesis. This thesis has not been submitted for the award of any other degree or thesis in any other institution.

All sentences or passages quoted in this thesis from other people's work have been specifically acknowledged by clear cross-referencing to author and work.

The submitted written version of the thesis corresponds to the electronic version.

Kiel, 26.01.2026

_____

Lauritz Timm

Abstract

The simulation of complex systems, like climate, weather and glaciers, is an important application of numerical methods.

Modeling such systems with real-world data requires extensive computations, resulting in a trade-off between high energy consumption and limited accuracy.

A state-of-the-art framework for modeling ice flow on a large scale is the *Ice Sheet and System Model* (ISSM), which is used to predict change and behavior of glaciers and ice sheets.

We look into how the existing implementation of Newton's method can be improved by adapting step-size control based on the Armijo condition.

To improve the overall user experience, we rework the compilation toolchain and investigate the configurability of ISSM following the paradigms of feature-oriented software design.

We find, that those concepts are not that easy to adapt in ISSM, but yield significant benefits for users and performance.

Contents

Acronyms

ISSM *Ice-sheet and Sea-level System Model*

GCC *GNU Compiler Collection*

ISMIP *Ice Sheet Model Intercomparison Project*

PETSc *Portable, Extensible Toolkit for Scientific Computation*

SOTA state-of-the-art

FOSD feature-oriented software development

IVP initial-value problem

API application programming interface

IDE integrated development environment

FM feature model

1 Introduction

Climate simulation and weather prediction are important topics of ongoing research, and an important application of numerical models [29].

A state-of-the-art (SOTA) framework for simulating ice flow is *Ice-sheet and Sea-level System Model* (ISSM), which uses a non-linear system of equations to predict surface velocities.

In this thesis, we adapt the work of Schmidt et al. [26], focusing on implementing Newton's method with a step-size satisfying the Armijo condition.

We want to show, that we can use easy-to-adapt approaches to improve performance of large numerical frameworks. This idea is based on feature-oriented software development (FOSD), which focuses on configurability of large software systems based on a feature model (FM).

Since we identified a development bottleneck in the compilation toolchain, we replace the exiting tools with CMake and other SOTA modules.

To evaluate our implementations we use the *Ice Sheet Model Intercomparison Project* (ISMIP) benchmarks, showing, that our novel approach outperform the existing algorithms.

The methodology is structured in three parts, focusing on FOSD, improvements of the toolchain, and algorithmic optimization.

1.1 Research Questions

To guide this thesis, we introduce the following research questions:

RQ1 How to the principles of FOSD apply to ISSM?

RQ1.1 Can we infer a FM for ISSM?

RQ2 What are prerequisites to understand a large numerical framework?

RQ2.1 How easy is it for a programmer to adapt to the code base?

RQ2.2 What pitfalls might be encountered?

RQ2.3 How much physical / mathematical background is required?

RQ3 How can we improve the user experience?

RQ3.1 Can we make configuration of ISSM more comprehensible?

RQ3.2 Can we speed up compilation using a SOTA toolchain?

RQ4 Can we improve the simulation performance of ISSM?

RQ4.1 What is the tradeoff of using Armijo step-size control?

RQ4.2 Do we converge faster with a good initial guess?

With RQ1 and RQ2 we focus on the software engineering of ISSM. We investigate how difficult it is for a software developer / computer scientist, without extensive mathematical or physical background, to comprehend, adapt and implement new parts of a numerical framework. In contrast, RQ3 aims at the common user, and how we can improve setting up and running the framework through improvements of the toolchain. Obviously this also benefits us during development and speeds up re-compilation of individual parts of the framework. Furthermore, we investigate for RQ4 how we can improve the approximation of Newton's method in ISSM and profit from our prior advances during development.

2 Background

In this section, we will introduce equations and concepts providing the underlying foundation for this paper.

2.1 Ice Sheets

The movement of ice sheets is driven by gravitation and is interlinked with temperature fluctuations and basal topography. With ongoing climate change strongly affecting polar regions [29], investigating the properties of continental glaciers is an important research endevour.

While field excursions yield results about the local thickness and surface properties, modeling ice-sheets give insight about ice-flow and internal dynamics of thick ice. Since ice is a non-Newtonian fluid [20], advanced numerical methods and frameworks are necessary to approximate those properties mathematically.

2.1.1 Ice Sheet System Model

ISSM is a finite element ice-flow model developed by the Jet Propulsion Laboratory and University of California at Irvine [4]. Since the source code is open-source, it can be easily used and adapted to custom modeling problems. The default configuration and build toolchain, using AUTOTOOLS, as described in the user manual [20], is shown in Fig. 1. It is important to note, that we include PETSc [12][6] as a library for scientific computations. The underlying mathematical and physical foundations will be introduced in section 2.6 and 2.7.

2.1.2 ISMIP

To compare the precision performance characteristics of different higher-order and full-Stokes ice-sheet models, we use the ISMIP benchmark [21]. This collection of six experiments aims to evaluate steady-state solutions of numerical approximations. For the scope of this thesis, we focus on the experiments ISMIP-A and ISMIP-B. Both experiments simulate ice flow over a bumpy bed. A slab of ice with thickness of 1 km lays on a sloping bed with an angle of $\alpha = 0.5°$. z_s and z_b are the surface and basal elevation respectively.

$$z_\mathrm{s}(x, y) = -x \tan(\alpha) \tag{1}$$

In addition, the basal topography is oscillating with $\omega = \frac{2\pi}{L}$. Let $x, y \in [0, L]$.

```
export ISSM_DIR="$(pwd)"
source .venv/bin/activate

autoreconf -ivf
./configure \
    --prefix=${ISSM_DIR} \
    --with-python-version=... \
    --with-python-dir=... \
    --with-python-numpy-dir=... \
    --with-fortran-lib=... \
    --with-mpi-include=... \
    --with-mpi-libflags=... \
    --with-metis-dir=... \
    --with-parmetis-dir=... \
    --with-blas-lapack-dir=... \
    --with-scalapack-dir=... \
    --with-mumps-dir=... \
    --with-petsc-dir=... \
    --with-triangle-dir=... \
    --with-chaco-dir=... \
    --with-m1qn3-dir=...

make -j24
make install -j24
```

Fig. 1: *Configuring and building ISSM with default configuration.*

For ISMIP-A we set the base to

$$z_{\mathrm{b}}(x, y) = z_{\mathrm{s}}(x, y) - 1000 + 500 \sin(\omega x) \sin(\omega y) \tag{2}$$

while the base of ISMIP-B is described by

$$z_{\mathrm{b}}(x, y) = z_{\mathrm{s}}(x, y) - 1000 + 500 \sin(\omega x) \tag{3}$$

We choose $L = 80\,\mathrm{km}$. Further details, especially constants for the numerical model will be introduced in section 2.1.2, as we implement the experiments for ISSM.

The geometries are shown in Fig. 2, the normed horizontal velocities $\|v_{\mathrm{s}}\|$ are compared along the red line on the surface at $y = \frac{L}{4}$. ISMIP provides the simulated values from several reference models, which we can use for comparison.

$$\|v_{\mathrm{s}}\| = \sqrt{v_x^2 + v_y^2} \tag{4}$$

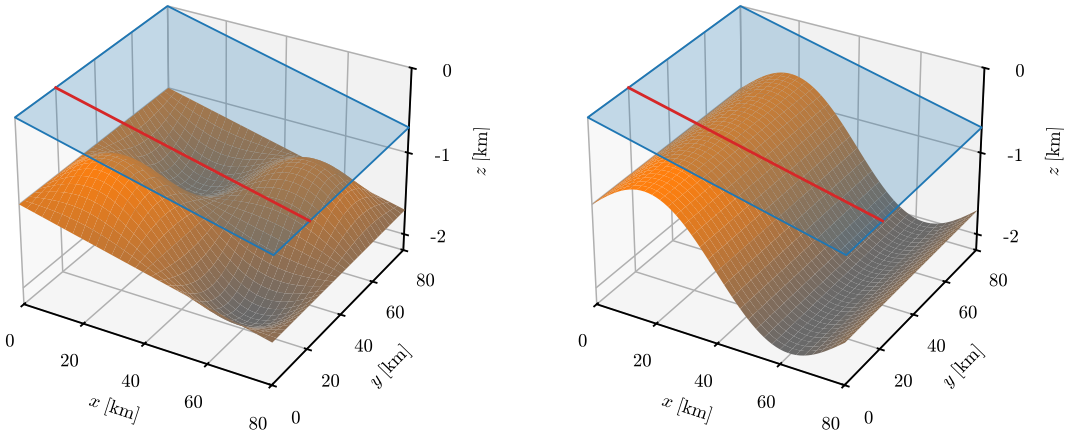

Fig. 2: *Geometry of experiments ISMIP-A (left) and ISMIP-B (right).*

2.2 Feature-oriented Software Development

Introduced as a paradigm for development of large-scale software systems, FOSD introduces the concept of a feature:

> "A feature is a unit of functionality of a software system that satisfies a requirement, represents a design decision, and provides a potential configuration option." [11]

With this definition, we can decompose the configuration options of a software system into a FM. A simple example for a car is shown in Fig. 3, including a selection of optional, mandatory, and mutually exclusive features. The gained insight about the relations of different components in such a model allows developers to introduce new features, without breaking existing dependencies.

2.3 C++ Language

The C++ programming language was developed as a high-level general-purpose programming language by Bjarne Stroustrup in 1985 [28]. Over its lifetime, the language had several extensions, which are described in the ISO C++ standard [14].

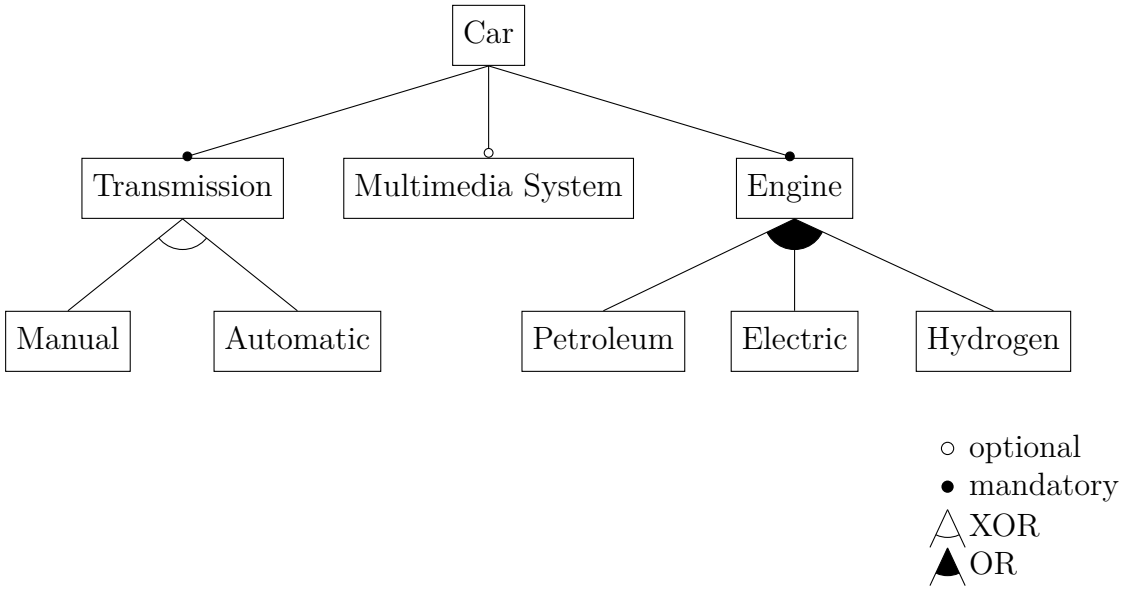

Fig. 3: *Simple feature-model of a car.*

2.3.1 Compiler

In contrast to other languages, like Java or Python, C++ source code needs to be translated into a binary representation, commonly the `x86-64` instruction set. This conversion is done by a compiler, for example *GNU Compiler Collection* (GCC) or Clang [10]. C++ has a robust type system, enabling the compiler to detect errors during processing. Furthermore, the compilation toolchain for C++ includes a preprocessor, which is executed before the actual compiler, which can be used to change certain parts of the source files.

2.3.2 Comparison with Python

A common language for entry-level programmers is Python, due to its easily comprehensible syntax [30]. Instead of being statically compiled into a binary, Python code is interpreted at runtime. Therefore, dynamic changes to the source code are reflected during execution. Unfortunately, Python does not have a robust type system, making it prone to errors from conflicting types. Compared to C++, Python performs worse, due to the overhead from translating code at runtime [30].

2.4 GCC / Autotools

GCC and GNU Autotools, constisting of Autoconf [18] and Automake [19], are part of the GNU toolchain. [3]. GNU Autotools is used to generate makefiles, which can then be processed using the compiler. Note, that since the toolchain is rule based, recursive includes and unintended dependencies might slow down the compilation significantly. While still commonly used, they fall short when compared to more modern tools, like CMake. For the scope of this thesis, we look into how we can replicate the logic of a GNU toolchain using SOTA replacements.

2.5 State-of-the-art Software Development

2.5.1 CMake

Similar to Autotools, CMake is meta-build tool to configure and generate build scripts for a compiler toolchain [15]. In contrast to Automake, Cmake does not only create makefiles, but uses a secondary data structure to identify dependencies between libraries and source files. Therefore rebuilding a project can be reduced to processing only a subset of source files, resulting in vastly improved performance.

In addition, integrated development environments (IDEs), like CLion, can use CMake to build a dependency tree, allowing a look-up of class and method definitions.

Running the CMake configuration creates UNIX Makefiles by default, but it also supports several other generators, in particular Ninja. In contrast to Autotools, the configuration files and compiled objects can be all build in a separate build-folder, encapsulating this process, and decoupling compiled sources from the file structure.

2.5.2 Ninja

Ninja is a modern build system, developed with a main focus on speed [5]. The manual lists four key design goals as guiding philosophy:

- very fast incremental builds
- support variety of build systems
- get dependencies correct
- prefer speed over convenience

Ninja can be natively used as generator for CMake.

2.5.3 LLVM / Clang

Originally designed to investigate dynamic compilation techniques, LLVM became the SOTA framework for compiler development [8] [16]. LLVM contains several components, most notably the Clang compiler [1], which, similar to GCC, aims to provide a compilation toolchain based on the C++ standard.

Due to its feature-richness for analyzing and processing code during compilation, many research projects use LLVM as framework to inject code into the compiler [27]. For example, the VARA toolsuite [25] is a research tool, developed for analyzing feature regions in large C++ projects.

For the scope of this thesis, we are interested in Clang as a replacement for GCC.

2.5.4 DUNE

DUNE is a similarSOTA framework to ISSM, primarily focusing on numerical calculation [2]. Similar to ISSM, it is written in C++ and can be used with a Python interface.

In contrast to ISSM, the source code of DUNE is extensively documented with automatic class documentation using Doxygen. Additional resources, like the *DUNE Cheat Sheet* [7] or a comprehensive book [24], support users in understanding the framework.

2.6 Mathematical Preliminaries

Let $\boldsymbol{f} : \mathbb{R}^n \to \mathbb{R}^m$ and a point $\boldsymbol{x} = (x_1, \ldots, x_n) \in \mathbb{R}^n$.

Define $\nabla = (\frac{\partial}{\partial x_1}, \ldots, \frac{\partial}{\partial x_n})$.

We get $\boldsymbol{f}(\boldsymbol{x}) = (f_1(\boldsymbol{x}), \ldots, f_m(\boldsymbol{x})) \in \mathbb{R}^m$.

Assume, that the first-order partial derivatives of f exist on $\mathbb{R}^n$.

$$\nabla f_1(\boldsymbol{x}) = \begin{bmatrix} \frac{\partial f_1(\boldsymbol{x})}{\partial x_1} \\ \vdots \\ \frac{\partial f_1(\boldsymbol{x})}{\partial x_n} \end{bmatrix} \quad \cdots \quad \nabla f_m(\boldsymbol{x}) = \begin{bmatrix} \frac{\partial f_m(\boldsymbol{x})}{\partial x_1} \\ \vdots \\ \frac{\partial f_m(\boldsymbol{x})}{\partial x_n} \end{bmatrix} \tag{5}$$

We define the Jacobian matrix $J_{\boldsymbol{f}}$:

$$J_{\boldsymbol{f}}(\boldsymbol{x}) = (\nabla \boldsymbol{f}(\boldsymbol{x}))^{\top} = \begin{bmatrix} \frac{\partial \boldsymbol{f}(\boldsymbol{x})}{\partial x_1} & \dots & \frac{\partial \boldsymbol{f}(\boldsymbol{x})}{\partial x_n} \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1(\boldsymbol{x})}{\partial x_1} & \dots & \frac{\partial f_1(\boldsymbol{x})}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m(\boldsymbol{x})}{\partial x_1} & \dots & \frac{\partial f_m(\boldsymbol{x})}{\partial x_n} \end{bmatrix} \tag{6}$$

2.6.1 Picard's Method

Given a initial-value problem (IVP)

$$\begin{cases} \frac{\partial \boldsymbol{x}(t)}{\partial t} = \boldsymbol{f}(\boldsymbol{x}(t), t) \\ \boldsymbol{x}(t_0) = \boldsymbol{x}_0 \end{cases} \tag{7}$$

we can compute the iteration

$$\boldsymbol{x}_1(t) = \boldsymbol{x}_0 \qquad \boldsymbol{x}_{k+1}(t) = \boldsymbol{x}_0 + \int_{t_0}^{t} \boldsymbol{f}(\boldsymbol{x}_k(s), s)\, \mathrm{d}s \tag{8}$$

such that this series converges against the solution $\boldsymbol{x}(t)$ [17].

2.6.2 Newton's Method

Let $\boldsymbol{x_k}, \boldsymbol{x_{k+1}} \in \mathbb{R}^n$. To find the root of a function, compute

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k - J_{\boldsymbol{f}}(\boldsymbol{x}_k)^{-1} \boldsymbol{f}(\boldsymbol{x}_k) \tag{9}$$

for a guess $\boldsymbol{x}_k$, such that, $\boldsymbol{x}_{k+1}$ is a better approximation of the root than $\boldsymbol{x}_k$ [13]. Instead, we can solve the system of linear equation.

$$J_{\boldsymbol{f}}(\boldsymbol{x}_k)(\boldsymbol{x}_{k+1} - \boldsymbol{x}_k) = -\boldsymbol{f}(\boldsymbol{x}_k) \tag{10}$$

with a solution

$$\boldsymbol{p}_k = \boldsymbol{x}_{k+1} - \boldsymbol{x}_k \tag{11}$$

so that we get the approximation

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \boldsymbol{p}_k \tag{12}$$

2.6.3 Armijo Condition

Let $\boldsymbol{x_k}, \boldsymbol{x_{k+1}} \in \mathbb{R}^n$, $\alpha \in \mathbb{R}$, and $\boldsymbol{p}_k$ the solution of Newtons's method from (11).

We compute

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \alpha \boldsymbol{p}_k \tag{13}$$

and want to minimize alpha, such that the inequality

$$\boldsymbol{f}(\boldsymbol{x}_k + \alpha \boldsymbol{p}_k)) \leq \boldsymbol{f}(\boldsymbol{x_k}) + \sigma\alpha \left((\nabla \boldsymbol{f}(\boldsymbol{x}_k))^\top \boldsymbol{p}_k \right) \tag{14}$$

still holds. Note, that

$$(\nabla \boldsymbol{f}(\boldsymbol{x}_k))^\top \boldsymbol{p}_k < 0 \tag{15}$$

always holds.

We can use the norm to compute a real solution. Because of (15), we need to change the sign on the right-hand side.

$$\|\boldsymbol{f}(\boldsymbol{x}_k + \alpha \boldsymbol{p}_k))\| \leq \|\boldsymbol{f}(\boldsymbol{x}_k)\| - \sigma\alpha \|(\nabla \boldsymbol{f}(\boldsymbol{x_k}))^\top \boldsymbol{p}_k\| \tag{16}$$

We call (16) the Armijo condition [13].

Fortunately, we see

$$\|(\nabla \boldsymbol{f}(\boldsymbol{x}_k))^\top \boldsymbol{p}_k\| = \|J_{\boldsymbol{f}}(\boldsymbol{x}_k)\boldsymbol{p}_k\| = \|-\boldsymbol{f}(\boldsymbol{x}_k)\| \tag{17}$$

which is the right-hand side of (10).

For the scope of this thesis, we choose $\|\cdot\| = \|\cdot\|_2$

2.7 Physical Preliminaries

2.7.1 Full-Stokes Equations

We write the conservation of momentum as

$$\rho \frac{\mathrm{d}\boldsymbol{v}}{\mathrm{d}t} = \nabla \cdot \sigma + \rho \boldsymbol{b} \tag{18}$$

with ice density ρ, velocity v, Cauchy stress tensor σ, and fore $\boldsymbol{b}$ [20]. If we ignore acceleration and the Coriolis effect, we can rewrite this formula as

$$0 = \nabla \cdot \sigma + \rho g \iff \nabla \cdot \sigma = -\rho g \tag{19}$$

We impose the stress tensor to be symmetrical, therefore

$$\sigma = \sigma^\top \tag{20}$$

In addition, we treat ice as a purely viscous incompressible material, such that

$$\sigma' = 2\mu\dot{\epsilon} \tag{21}$$

with the deviatoric stress tensor σ', ice viscosity μ, and strain rate $\dot{\epsilon}$ [20].

As ice is an non-Newtonian fluid, we describe its viscosity with the generalized Glen's flow law [20].

$$\mu = \frac{B}{2\dot{\epsilon}_{\mathrm{e}}^{\frac{n-1}{n}}} \tag{22}$$

B is the ice hardness, n is Glen's flow law exponent, and $\dot{\epsilon}_{\mathrm{e}}$ is the effective strain rate, as defined through

$$\dot{\epsilon}_{\mathrm{e}} = \sqrt{\frac{1}{2}\sum_{i,j}\dot{\epsilon}_{ij}^2} = \frac{1}{\sqrt{2}}\|\dot{\epsilon}\|_{\mathrm{F}} \tag{23}$$

2.7.2 Higher-Order Equations

The higher-order model of ISSM assumes bridging effects are neglectable, as well as the horizontal gradient of vertical velocities compared to vertical gradients of horizontal velocities. [20]

Therefore, the Full-Stokes equations for the surface s can be reduced to two equations with two unknowns

$$\begin{cases} \nabla \cdot (2\mu\dot{\epsilon}_{\mathrm{HO}_x}) = \rho g \frac{\partial s}{\partial x} \\ \nabla \cdot (2\mu\dot{\epsilon}_{\mathrm{HO}_y}) = \rho g \frac{\partial s}{\partial y} \end{cases} \tag{24}$$

with

$$\dot{\epsilon}_{\mathrm{HO}_x} = \begin{bmatrix} 2\frac{\partial v_x}{\partial x} + \frac{\partial v_y}{\partial y} \\ \frac{1}{2}\left(\frac{\partial v_x}{\partial y} + \frac{\partial v_y}{\partial x}\right) \\ \frac{1}{2}\frac{\partial v_x}{\partial z} \end{bmatrix} \qquad \dot{\epsilon}_{\mathrm{HO}_y} = \begin{bmatrix} \frac{1}{2}\left(\frac{\partial v_x}{\partial y} + \frac{\partial v_y}{\partial x}\right) \\ \frac{\partial v_x}{\partial x} + 2\frac{\partial v_y}{\partial y} \\ \frac{1}{2}\frac{\partial v_y}{\partial z} \end{bmatrix} \tag{25}$$

3 Improving Simulation Performance

3.1 Feature Models for ISSM

In this section, we construct FMs for ISSM, to get insights of the configuration space. Since the framework is configurable during both compile- and run-time, we need to discuss those aspects individually.

3.1.1 Compile-time Features

The first model consists of preprocessor directives and toolchain switches, enabling or including certain external libraries or parts of ISSM. We sub-divide this model into high level configurability affecting general composition of the library sources including external libraries, and lower options, replacing only parts of the source code.

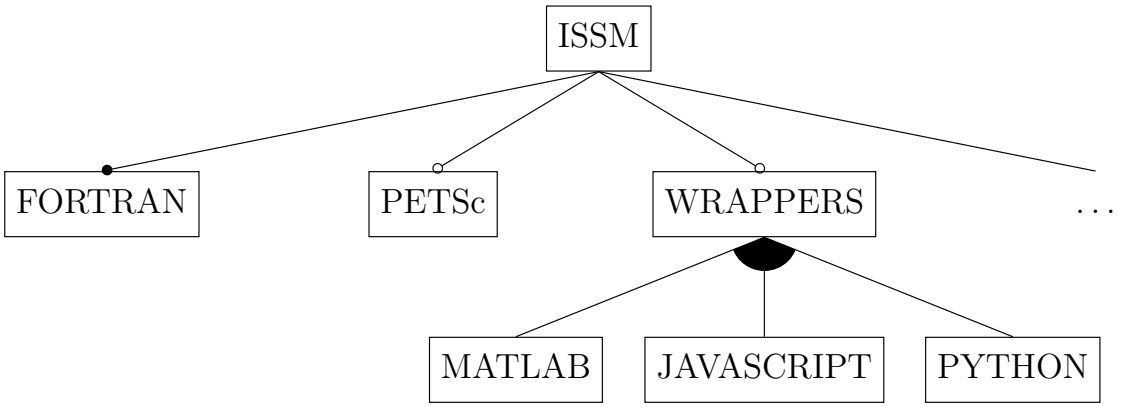

Fig. 4: *High level feature model of ISSM.*

Fig. 4 shows an except of the high-level FM. We see, that in addition to the mandatory inclusion of fortran libraries, we can choose, for example, to use the numerics library *Portable, Extensible Toolkit for Scientific Computation* (PETSc). Moreover, we can decide to build any number of wrappers for the framework, especially python bindings, which we use for our own implementation.

We will see in section 3.2, how we configure and process those options during compilation. For the scope of this section, we can assume, that enabling an option on high level influences on a lower level, which source files are used, including additions and modification to the existing source code.

Therefore, on the lower level, we need to look into specific parts of the implementation. Some features replace other features, for example including the library PETSc, replaces parts of ISSM.

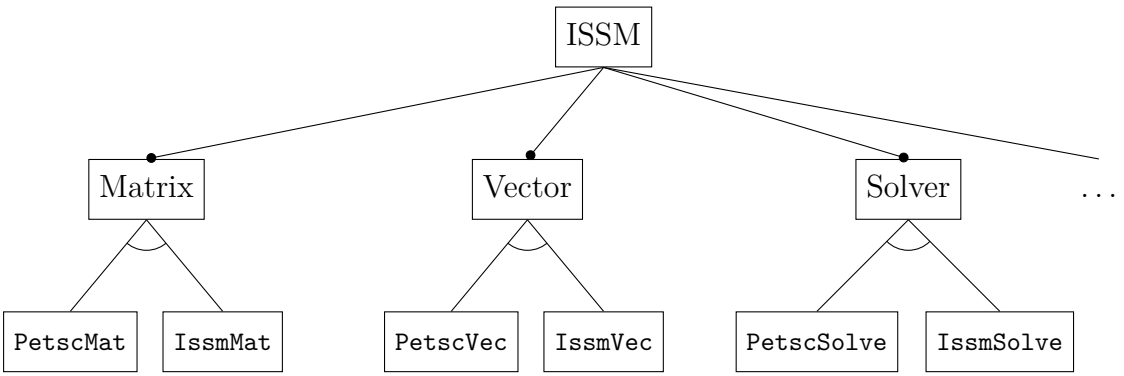

Fig. 5: *Low level feature model with PETSc.*

This is visualized in the second FM Fig. 5, showing exemplary parts affected by including PETSc. For several components of the linear algebra classes, we replace certain parts of the default implementation, for example `IssmMat`, with corresponding classes of PETSc, in this case `PetscMat`. This keeps the implementation consistent, such that all occurrences of the default implementations are now replaced by the library. Note, that those choices are mutually exclusive, and there exists cross-constraints, not visualized in the FM, such that one has to choose either choose to use default implementations everywhere or replace every occurrence with PETSc to keep the source code consistent.

```
#ifdef _HAVE_PETSC_
PetscMat<doubletype> *pmatrix;
#endif
IssmMat<doubletype> *imatrix;
```

```
#ifdef _HAVE_PETSC_
PetscVec<doubletype> *pvector;
#endif
IssmVec<doubletype> *ivector;
```

Fig. 6: *Enabling PETSc matrices and vectors.*

If we look into how those switches are implemented, we see in Fig. 6, that a preprocessor definition of the variable `PETSC` conditionally replaces parts of the datatypes

and variables with either the default or library version.

```
void MatMult(Vector<doubletype> *X, Vector<doubletype> *AX) {
    if (type == PetscMatType) {
#ifdef _HAVE_PETSC_
        this->pmatrix->MatMult(X->pvector, AX->pvector);
#endif
    }
    else {
        this->imatrix->MatMult(X->ivector, AX->ivector);
    }
}
```

Fig. 7: *Computing with PETSc matrices.*

In addition to updating the allocations, we also need to adapt the code, to work with either datatype. Exemplary, we look into the multiplication of matrices. Fig. 7 shows the two parts of the implementation, handling either `PetscMat` or `ISSMMat`.

```
switch (Kff->type) {
ef _HAVE_PETSC_
    case PetscMatType: {
        ...
        PetscSolve(&uf->pvector, Kff->pmatrix, pf->pvector, ...);
        break;
    }
if
    case IssmMatType: {
        IssmSolve(&uf->ivector, Kff->imatrix, pf->ivector, ...);
        break;
    }
    default:
        ...
}
```

Fig. 8: *Enabling PETSc solver.*

Furthermore, we can observe this behavior if we look into a more sophisticated example. Fig. 8 shows, that with PETSc, we use the included solver.

Since most of those options are conditionally enabled through a preprocessor directive, we assure, that on one hand we only build one part of the source code, and

on the other hand detect inconsistencies in the inclusion of the library through the static type analysis. Since all operations work on only one of those distinct options, the compiler can directly reject code with seemingly unknown datatypes.

3.1.2 Run-time Features

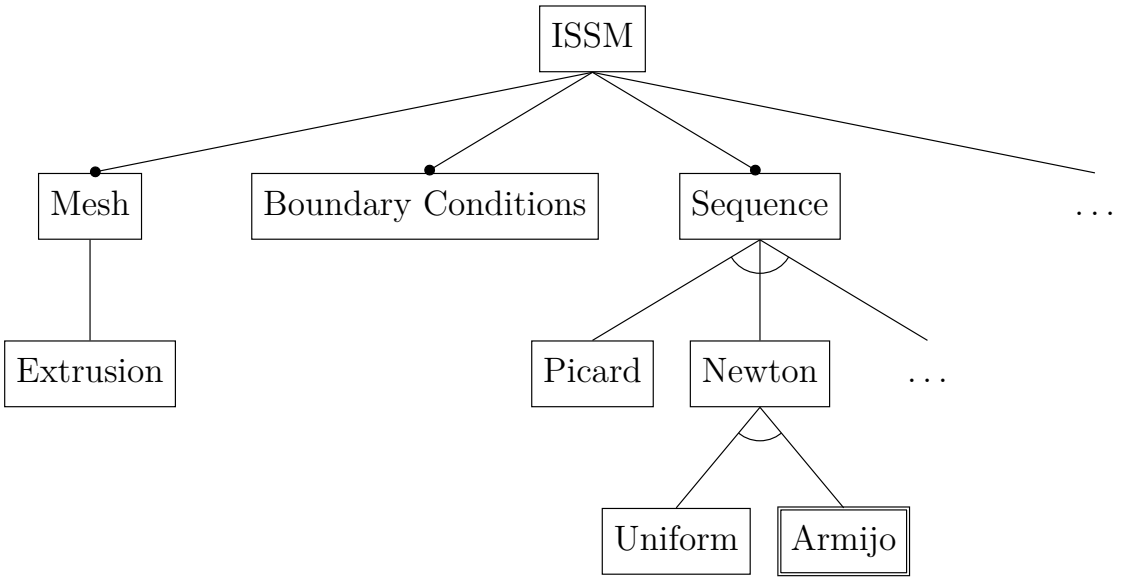

Fig. 9: *Run-time feature-model of ISSM.*

In addition to modifying the framework, ISSM has a vast number of configuration options for simulations / experiments, plotting and data processing. We include a small excerpt of the FM in Fig. 9, focusing on parts relevant to this thesis. For, example the user can adapt the mesh and number of extrusion layers, affecting the precision of a simulation. This will be especially interesting, as we look into the tradeoff between mesh density and computation time, as finer grids require more time to process completely. We included boundary conditions, as they are a vital part of our benchmarks, see section 2.1.2. Choosing a solver for approximation might drastically change the results of the simulation. For the second part of this thesis, focusing on improving the approximation algorithm of ISSM, we provide a new sub-feature to use Newton's method with step-size control, highlighted red in Fig. 9.

3.2 Building with CMake

ISSM is natively build with AUTOMAKE, resulting in an outdated compilation toolchain. The benefits of using CMAKE over AUTOMAKE will be discussed in section 5, for this section we focus on how we adapted and replicated the build logic.

3.2.1 Setup

To initialize CMAKE, we need to set several parameters, as shown in Fig. 10 in the top-level `CMakeList.txt`. Note, that we need to enable FORTRAN support for building certain parts of the framework. CMAKE allows us to specify where we want the build binary. To mimic the old behavior of ISSM, we set the output directory to `bin` and `lib`, nested directly in the source root folder.

```
cmake_minimum_required(VERSION 3.13)

project("ISSM" CXX C Fortran)
enable_language(Fortran)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)
set(CMAKE_EXPORT_COMPILE_COMMANDS YES)
set(CMAKE_POSITION_INDEPENDENT_CODE ON)

add_compile_definitions(ISSM_PREFIX="...")

set(CMAKE_RUNTIME_OUTPUT_DIRECTORY ".../bin")

set(CMAKE_STATIC_LIBRARY_PREFIX "")
set(CMAKE_SHARED_LIBRARY_PREFIX "")
set(CMAKE_LIBRARY_OUTPUT_DIRECTORY ".../lib")
set(CMAKE_ARCHIVE_OUTPUT_DIRECTORY ".../lib")
```

Fig. 10: *Initilization of CMAKE.*

3.2.2 Configuration

We note, that most of the library source files start with a header guard.

```
#ifdef HAVE_CONFIG_H
    #include <config.h>
#else
#error ...
#endif
```

We see, that the source code has a hard dependency on the value of `HAVE_CONFIG_H` and subsequently `config.h`. This included header file `config.h` and definition of `HAVE_CONFIG_H` is generated during execution of the configuration script produced by AUTOTOOLS. Since we remove this part of the toolchain, we have to mimic this behavior.

If we look into the configuration header, we see, that this file defines macros, such as `_HAVE_PETSC_`, as seen in Fig. 6, enabling features. Therefore, this is an integral part of the library and has to be carefully adapted.

To guide this process, we generate an initial `config.h` with AUTOTOOLS, and move it to a new `config` folder. Note, that those preprocessor definitions only affect parts of the source code, not the toolchain. To address this issue, we build a second configuration file `config.cmake`, which enables the feature in the first place, for example PETSC, for CMAKE.

```
add_compile_definitions(HAVE_CONFIG_H)
set(PETSC true)
```

Note, that those options can also be set by passing the option directly to the CMAKE executable, very similar to the syntax of AUTOTOOLS configuration script.

We always define `HAVE_CONFIG_H` in our top-level `CMakeList.txt` for the compiler, so that the header guard always works. We include the configuration folder on the top level of CMAKE.

```
list(APPEND CMAKE_MODULE_PATH ".../config")
include(config)
include_directories(config)
```

Since CMAKE allows us to add compile definitions, we remove `_HAVE_PETSC_` from the configuration header, and instead set it during configuration with CMAKE.

```
if (PETSC)
    add_compile_definitions(_HAVE_PETSC_)
    add_compile_definitions(HAVE_MPI)
    add_compile_definitions(_HAVE_METIS_)
endif ()
```

We only investigate a single configuration of ISSM, hence the configuration header and CMake configuration might need to be adapted by hand for other use cases. We discuss in section 5 how this can be improved upon by extending our concept to handle all possible configurations.

In addition, we can use this entry point, to handle the detection, inclusion and linking of external libraries.

```
if (PETSC)
    include_directories(...)
    link_directories(...)
    list(APPEND ISSM_EXTERNAL_LIBS petsc mpi metis)
endif ()
```

We can define a target for CMake. This is the resulting executable `issm.exe`.

```
add_subdirectory(src)
add_executable(issm.exe src/c/main/issm.cpp)
target_link_libraries(issm.exe ${ISSM_EXTERNAL_LIBS} libISSMCore ...)
```

Note, that we include the `src` directory, recursively with other `CMakeList.txt` on lower levels.

If we look at the generation files for Autotools, we see that the toolchain conditionally builds up a list of source files and links them in a library object.

```
if PETSC
issm_sources += \
    ...
    ./toolkits/petsc/objects/PetscMat.cpp \
    ./toolkits/petsc/objects/PetscVec.cpp \
    ./toolkits/petsc/objects/PetscSolver.cpp
endif
...
libISSMCore_la_SOURCES = $(issm_sources)
libISSMCore_la_LDFLAGS += -static
libISSMCore_LIB_ADD = $(MATHLIB) $(FORTRANLIB) $(PETSCLIB) ...
```

We replicate the same logic semantically equivalent in CMake.

```
if (PETSC)
    list(APPEND ISSM_SOURCES
        ...
        toolkits/petsc/objects/PetscMat.cpp
        toolkits/petsc/objects/PetscVec.cpp
        toolkits/petsc/objects/PetscSolver.cpp
    )
endif ()
...
add_library(libISSMCore STATIC ${ISSM_SOURCES})
target_link_libraries(libISSMCore m gfortran petsc ...)
```

3.2.3 Python Bindings

Next, we need to look into building Python bindings for ISSM, which are used in an extensive application programming interface (API) to initialize, parametrize and visualize experiments.

Since we want to keep our implementation as close as possible to the original toolchain, we do use the existing framework to build bindings.

Similar to the behavior of Autotools, we conditionally include the corresponding subdirectory. For the scope of this thesis, we limit to Python, while ISSM originally also supports JavaScript and Matlab integration.

```
if (WRAPPERS AND PYTHON)
    add_subdirectory(python)
endif ()
```

The bindings themselves are similarly build to the main source files, we replicate the logic in CMake.

Autotools sets a global variable `PACKAGE_BUILD_DATE` during configuration. Since the only usage of this variable occurs to print the compilation time in the Python API, we remove the dynamic calculation from the top-level and only include at at this level. Fortunately, we can use a CMake build-in command to get the appropriate timestamp.

```
string(TIMESTAMP PACKAGE_BUILD_DATE "%a-%b-%d-%H:%M:%S-%Z-%Y")
target_compile_definitions(... PACKAGE_BUILD_DATE="${PACKAGE_BUILD_DATE}")
```

In addition to the linked libraries, the AUTOTOOLS toolchain also copies the PYTHON API scripts into the `bin` folder, so we replicate this in CMAKE.

```
file(GLOB BIN_SCRIPTS
    ...
)
foreach (ORIGINAL IN ITEMS ${BIN_SCRIPTS})
    get_filename_component(LINK "${ORIGINAL}" NAME)
    file(CREATE_LINK "${ORIGINAL}" ...)
endforeach ()
```

As we did not see the need to copy those files each run, as done by AUTOTOOLS, we only create softlinks. This has the advantage that changes to the scripts in `bin` are not overwritten, as we did change the original source file.

3.2.4 Running CMake

With our toolchain complete, we can run CMAKE with NINJA and build our library. The build object files will be located in the `build` folder, while binaries and shared libraries can be found in `bin` and `lib` respectively, conceptually identical to the behavior of AUTOTOOLS. Note, that we use a PYTHON virtual-environment in `.venv` to handle python dependencies.

```
export ISSM_DIR="$(pwd)"
source .venv/bin/activate

mkdir -p build
cd build || exit

cmake -GNinja .. --fresh
cmake --build . -j24
```

Note, that we can easily switch out NINJA with MAKE by replacing the corresponding command with

```
cmake -G "Unix-Makefiles" .. --fresh
```

In addition we can also use CLANG instead of GCC by setting the appropriate environment variables before calling CMAKE.

```
export CC=/usr/bin/clang
export CXX=/usr/bin/clang++
```

3.3 Implementing Step-size Control

With the improved toolchain, we can address the implementation of Armijo step-size control for Newton's method. We refer to this approach as Armijo-Newton method.

This approach needs to be implemented as a solution sequence in ISSM. The source code already includes an implementation of Picards's method, which is the default. ISSM already includes a naive implementation of Newton's method, which we adapt to use the benefits of adaptive step-sizes. Fortunately, the existing code serves as guide to insert the required logic.

After allocating several matrices, the main part is executed in loop, calculating incrementally better approximation of the velocity.

The current velocity is stored in `uf`. We duplicate this value, to remember it for later usage.

```
uf = old_uf->Duplicate();
old_uf->Copy(uf);
```

Next, we calculate the current approximation of the velocity. Note, that the assignment to `pJf` in the first line only serves to tell the framework, to reserve the required space for the length of this vector. The actual function is represented through the stress tensor stored in `Kff`. Multiplying this with the current velocity yields the intended result, which is stored in `pJf`.

```
pJf = pf->Duplicate();
Kff->MatMult(uf, pJf);
```

Since we want to solve the system of linear equations for, as described in (10), we need to prepare the right-hand side. According to (19) we are only interested in the gradient for the velocity, but know the influence of pressure `pf` on the solution, so that we can simply subtract this vector from `pJf`. We then multiply a negative sign onto `pJf`, to construct the right-hand side of (10).

```
pJf->AXPY(pf, -1.0);
pJf->Scale(-1.0);
```

The main part of the sequence is done by calculating the current Jacobi matrix `Jff` of the system, and subsequently solving (10). This derivation is done by ISSM for the higher-order model and stress-balance analysis. Note, that `Solverx` dispatches to a suitable implementation, for example `KSPSolve` from the PETSc library.

```
CreateJacobianMatrixx(&Jff, femmodel, kmax);
Solverx(&duf, Jff, pJf, nullptr, nullptr, femmodel->parameters);
```

The solution of this process is stored in `duf` and can be used to calcuate a new velocity `uf`. But first, we need to take advantage of (17), as we already know the right-hand side of (10).

```
IssmDouble gradient = pJf->Norm(NORM_TWO);
```

Now we no longer need the value in `pJf` and reuse the space to computate the old velocity, which is used for comparison in the Armijo condition. Additionally we set the initial step width `alpha` to one, and `gamma` to an appropriate value as discussed by Schmidt et al. [26].

```
IssmDouble alpha = 1.0;
IssmDouble gamma = 1e-10;

old_uf->Copy(uf);
uf->AXPY(duf, alpha);
Kff->MatMult(uf, pJf);

IssmDouble min_term_old = pJf->Norm(NORM_TWO);
IssmDouble min_term_new;
```

With most pre-requirements done, we can try to find a minimal step-size. This is done in a loop, which calculates the norm of the velocity with a new step-size, decreasing `alpha` until the Armijo condition is fulfilled.

```
for (;;) {
    old_uf->Copy(uf);
    uf->AXPY(duf, alpha);
    Kff->MatMult(uf, pJf);

    min_term_new = pJf->Norm(NORM_TWO);

    if (min_term_new <= min_term_old - gamma * alpha * gradient) {
        break;
    }

    alpha = .5 * alpha;
}
```

Since we do not change any other parts of the code and our approximation is stored in `uf`, identical to the original sequence, we do not change the rest of this file. The velocity now gets propagated to the underlying model.

3.4 ISMIP for ISSM

To evaluate ISSM, including our new algorithm, we benchmark the framework with the experiments ISMIP-A and ISMIP-B, see section 2.1.2.

3.4.1 Parametrization

This is done exclusively by using ISSM Python API and calling the required command from the framework. Fortunately, the ISSM user guide [20] already has an example implementation of the ISMIP-A experiment. We use this code as guidance for our own implementation.

First, we need to load the ISSM libraries and initialize an empty model.

```
import numpy as np

ISSM_DIR = Path(...).absolute()

sys.path.append(str(ISSM_DIR / 'bin'))
sys.path.append(str(ISSM_DIR / 'lib'))
sys.path.append(str(ISSM_DIR / 'share'))
sys.path.append(str(ISSM_DIR / 'share/proj'))

from issmversion import issmversion

from model import *
from squaremesh import squaremesh
from plotmodel import plotmodel
from export_netCDF import export_netCDF
from loadmodel import loadmodel
from setmask import setmask
from parameterize import parameterize
from setflowequation import setflowequation
from socket import gethostname
from solve import solve

md = model()
```

Next we model the rough parameters for the simulation, especially the mesh and the number of extrusion layers. Note, that the ISSM Python API always operates on a model object, in this case `md`, and applies changes in-place, but also returns the same model with changed parameters. For comparison, we choose 80 km as dimensions for the ice-sheet, and create a mesh with 65 nodes. Instead of 33 nodes, as suggested by the example, we use a finer grid to improve accuracy.

```
L = 80_000
md = squaremesh(md, L, L, 65, 65)
md = setmask(md, '', '')
```

Next, the example implementation loads a parameter file. As we will discuss in section 5, this is ill-advised, and we will simply add the nested parameters to this script.

```
md.miscellaneous.name = 'IsmipA_cor'

md.geometry.surface = -md.mesh.x * np.tan(0.5 * np.pi / 180.0)
L = np.nanmax(md.mesh.x) - np.nanmin(md.mesh.x)
md.geometry.base = md.geometry.surface - 1000.0 + 500.0
    * np.sin(md.mesh.x * 2.0 * np.pi / L)
    * np.sin(md.mesh.y * 2.0 * np.pi / L)
md.geometry.thickness = md.geometry.surface - md.geometry.base

md.friction.coefficient = 200.0 * np.ones((md.mesh.numberofvertices))
md.friction.p = np.ones(md.mesh.numberofelements)
md.friction.q = np.ones(md.mesh.numberofelements)
```

Note, that we construct the experiment ISMIP-B absolutely identical to this, except we need to remove the last factor for the base computation.

```
md.geometry.base = md.geometry.surface - 1000.0 + 500.0
    * np.sin(md.mesh.x * 2.0 * np.pi / L)
```

In both cases, we replace the built-in constants of ISSM with the correct values, as specified by ISMIP. [21] Note, that the provided example does not do that, but uses the default values, and therefore simulates slightly different properties.

```
md.materials.rho_ice = 910
md.materials.rheology_B = 6.808188376e7
    * np.ones(md.mesh.numberofvertices)
md.materials.rheology_n = 3 * np.ones(md.mesh.numberofelements)

md = SetIceSheetBC(md)
```

Again, we deviate from the example, and use a higher vertical resolution of 12 extrusion layers.

```
md = md.extrude(12, 1)
md = setflowequation(md, 'HO', 'all')
```

Next we need to fix the boundary conditions. This is done like in the example by setting velocity at the base of the ice-sheet to zero and implementing periodic border-conditions, simulating an infinite spanning geometry.

```
md.stressbalance.spcvx = np.nan * np.ones(md.mesh.numberofvertices)
md.stressbalance.spcvy = np.nan * np.ones(md.mesh.numberofvertices)

basalnodes = np.nonzero(md.mesh.vertexonbase)
md.stressbalance.spcvx[basalnodes] = 0.0
md.stressbalance.spcvy[basalnodes] = 0.0

md.stressbalance.vertex_pairing = ...

export_netCDF(md, "ISMIP-BoundaryCondition.nc")
```

Most importantly, we need to tell ISSM to use Newton's method instead of Picard iteration for solving the non-linear problem. We extended the configuration of ISSM, such that setting `isnewton` to three enables our algorithm.

```
md = loadmodel("ISMIP-BoundaryCondition.nc")
md.cluster = generic('name', gethostname(), 'np', 2)

md.stressbalance.isnewton = 3
md.stressbalance.maxiter = ...
```

Lastly we need to call the `solve` method and store the results afterwards.

```
md = solve(md, 'Stressbalance')

export_netCDF(md, "ISMIP-StressBalance.nc")
```

Note, that the example implementation of this benchmark makes an error by using the length of the velocity vector in all dimensions for comparison.

```
v = md.results.StressbalanceSolution.Vel
```

This is not correct according to the specification of ISMIP [21]. Instead, we get only the horizontal velocities and compare the norm against other solutions, as provided by ISMIP [21].

```
vx = md.results.StressbalanceSolution.Vx
vy = md.results.StressbalanceSolution.Vy
```

3.4.2 Running Benchmark

To run the constructed script we execute

```
ISSM_DIR="$(pwd)"
export ISSM_DIR

source .venv/bin/activate
source etc/environment.sh

python ...
```

4 Evaluation and Results

In this section, we evaluate and show the results of our improvements to ISSM.

4.1 Building Faster with CMake

First, we ran a clean installation with all toolchains, expecting a full build of all objects.

Next, we use the `touch` command to update the timestamp of a source file. We conveniently choose part of the code, which was changed by us file during development of Newton's method.

To measure the performance we subsequently run the toolchain to investigate tool behavior regarding unchanged sources.

AUTOTOOLS takes around 20 minutes to configure, build and link the library from around 500 objects. We see a slight improvement, if we use a faster compiler, in this case CLANG. Notably AUTOTOOLS does not have a concept to detect unchanged source files, such that we need to rebuild and relink all object files for each subsequent run.

Tab. 1, visualized as stacked bar-chart in Fig. 11, show, that we can significantly reduce the amount of time spent for either initial or subsequent compilations. We observed, that we can clean configure, build and link the whole library, consisting of 531 targets, in around three minutes. For subsequent run, the new toolchains take advantage of rebuilding selectively the updated file and libraries or objects dependent on it, accumulating to only 23 targets, reducing the total time to under a minute.

4.2 Faster Convergence with step-size-control

To evaluate the precision and performance properties of our new solution sequence, we simulate the ISMIP benchmark and compare them to the reference values. The setup of our evaluation is similar to the work of Schmidt et al. [26].

The relative difference to the reference values was received as

$$\sqrt{\frac{\sum_x |v_x - v_{x_{\text{ref}}}|^2}{\sum_x |v_x|^2}} \tag{26}$$

for all simulated surface points at $y = \frac{L}{4}$, as shown in Fig. 2.

In addition to Picard's method, which is the default solution sequence of ISSM, we ran our step-size controlled Armijo-Newton Method.

	Toolchain	Configuration [s]	Build [s]	Install [s]	∑ [s]
Initial	Autotools(GCC)	23.99 ± 0.15	522.62 ± 1.67	669.08 ± 2.52	1226.75 ± 0.70
	Autotools(Clang)	22.63 ± 1.25	361.80 ± 6.56	662.57 ± 9.73	1060.23 ± 14.28
	CMake(GCC, Make	18.49 ± 1.06	166.37 ± 2.82	–	184.87 ± 3.34
	CMake(Clang, Make)	22.26 ± 0.33	136.99 ± 1.36	–	159.25 ± 1.06
	CMake(GCC, Ninja	21.04 ± 0.75	158.04 ± 3.31	–	179.08 ± 3.79
	CMake(Clang, Ninja)	27.08 ± 0.37	131.13 ± 0.60	–	158.21 ± 0.24
Update	Autotools(GCC)	16.61 ± 0.42	519.19 ± 0.20	668.41 ± 4.31	1204.22 ± 4.94
	Autotools(Clang)	14.99 ± 0.90	358.25 ± 1.26	637.73 ± 48.75	1010.98 ± 50.03
	CMake(GCC, Make	11.92 ± 1.06	90.78 ± 1.92	–	102.70 ± 1.42
	CMake(Clang, Make)	11.75 ± 0.13	86.21 ± 1.22	–	97.96 ± 1.09
	CMake(GCC, Ninja	11.25 ± 0.69	39.57 ± 1.05	–	50.82 ± 1.03
	CMake(Clang, Ninja)	13.65 ± 0.73	37.93 ± 0.00	–	51.59 ± 0.74

Tab. 1: *Mean compilation time for toolchains.*

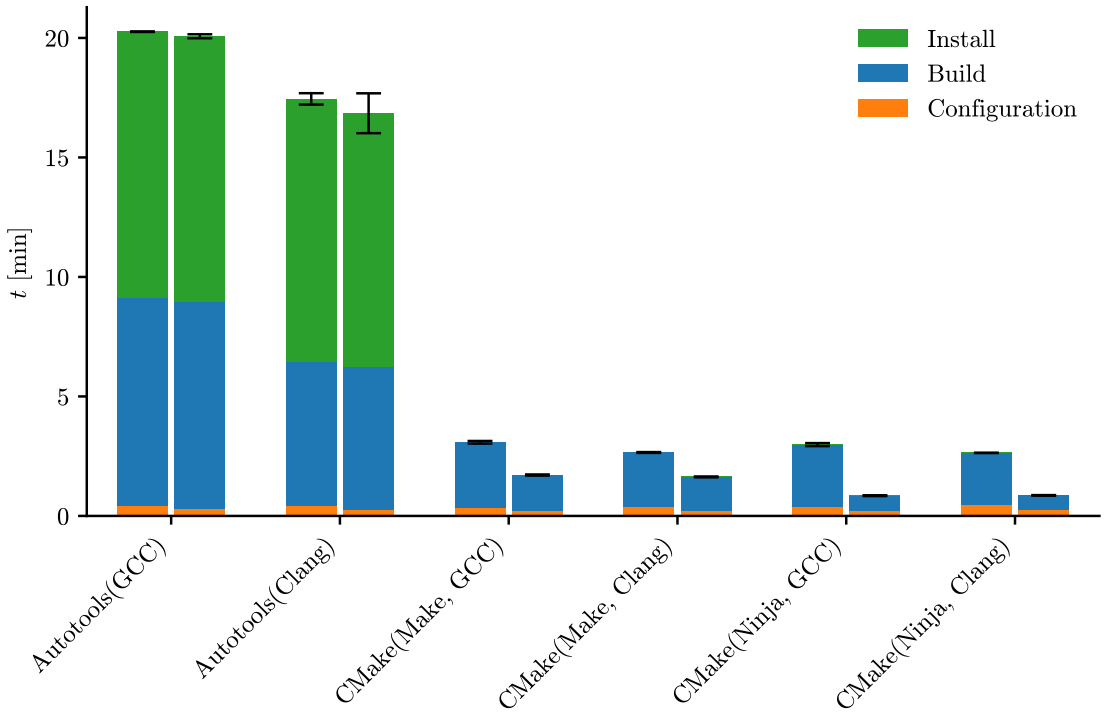

Fig. 11: *Mean compilation time for toolchains for initial build (left) and update (right).*

	Sequence	Steps	Solver [s]	Jacobi [s]	Armijo [s]	Total [s]
ISMIP-A	Picard	24	605.99 ± 11.53	–	–	748.19 ± 12.86
	Picard → Armijo Newton	18	492.16 ± 1.76	84.50 ± 0.70	0.07 ± 0.00	691.54 ± 4.04
	Armijo Newton	19	495.88 ± 1.48	88.64 ± 1.01	0.07 ± 0.00	698.06 ± 1.34
ISMIP-B	Picard	25	640.58 ± 2.23	–	–	787.77 ± 3.67
	Picard → Armijo Newton	17	472.89 ± 2.21	79.97 ± 0.42	0.06 ± 0.00	660.89 ± 2.12
	Armijo Newton	18	476.68 ± 2.95	83.52 ± 0.68	0.07 ± 0.00	667.19 ± 3.99

Tab. 2: *Mean execution time of solution sequences..*

As described by Pollock et al. [23], Newton's Method benefits from an good initial guess. Therefore we evaluate a third solution sequence, consisting of a single initial Picard iteration followed by the Armijo-Newton Method.

4.2.1 ISMIP-A

We ran the ISMIP-A experiment with each solution sequences for 25 steps. The resulting values are listed Tab. 3.

A stepwise visualization of the solution sequence is presented in Fig. 12 We see, that while Newton's method initially performs worse than both other methods, all solution quickly converge to a static solution. At 25 iteration the prediction of each solution sequence is indistinguishable from the expected values.

In addition, we visualized the convergence of the velocities in Fig. 14. Again, we observe that all solution converge quickly, but the sequences using step-size control outperform Picard's Method.

If we compare the estimated points, at which each solution converges, we see in Tab. 2, that while the advantage of an initial guess iteration is negligible, both Armijo based approaches are significantly faster than Picard's Method. While calculating the Jacobi matrix takes around five seconds per iteration, both methods approach a solution roughly 50 seconds faster.

4.2.2 ISMIP-B

Similar to the values observed for ISMIP-A, the Armijo approaches outperform the default solution sequence. Visualized in Fig. 12 and Fig. 14, the step-size controlled solution show an even faster convergence.

Looking at the times, the difference between Picard's method and Armijo-Newton extends to over a minute of time gained, due to a better approximation. Looking

Step	Δ ISMIP-A $\left[\frac{\mathrm{m}}{\mathrm{a}}\right]$			Δ ISMIP-B $\left[\frac{\mathrm{m}}{\mathrm{a}}\right]$		
	Picard	Picard ↓ Armijo Newton	Armijo Newton	Picard	Picard ↓ Armijo Newton	Armijo Newton
1	3.78×10^{-1}	3.78×10^{-1}	6.84×10^{-1}	4.15×10^{-1}	4.15×10^{-1}	7.03×10^{-1}
2	2.95×10^{-1}	2.67×10^{-1}	4.96×10^{-1}	3.32×10^{-1}	2.96×10^{-1}	5.17×10^{-1}
3	2.15×10^{-1}	1.56×10^{-1}	3.11×10^{-1}	2.48×10^{-1}	1.77×10^{-1}	3.29×10^{-1}
4	1.50×10^{-1}	8.38×10^{-2}	1.77×10^{-1}	1.79×10^{-1}	9.86×10^{-2}	1.91×10^{-1}
5	1.03×10^{-1}	4.31×10^{-2}	9.41×10^{-2}	1.27×10^{-1}	5.38×10^{-2}	1.05×10^{-1}
6	6.93×10^{-2}	2.19×10^{-2}	4.83×10^{-2}	8.87×10^{-2}	3.01×10^{-2}	5.71×10^{-2}
7	4.63×10^{-2}	1.21×10^{-2}	2.45×10^{-2}	6.20×10^{-2}	1.81×10^{-2}	3.18×10^{-2}
8	3.09×10^{-2}	8.81×10^{-3}	1.34×10^{-2}	4.37×10^{-2}	1.24×10^{-2}	1.90×10^{-2}
9	2.09×10^{-2}	8.25×10^{-3}	9.33×10^{-3}	3.12×10^{-2}	9.80×10^{-3}	1.29×10^{-2}
10	1.47×10^{-2}	8.35×10^{-3}	8.42×10^{-3}	2.29×10^{-2}	8.67×10^{-3}	1.01×10^{-2}
11	1.12×10^{-2}	8.49×10^{-3}	8.41×10^{-3}	1.75×10^{-2}	8.17×10^{-3}	8.81×10^{-3}
12	9.47×10^{-3}	8.59×10^{-3}	8.52×10^{-3}	1.39×10^{-2}	7.94×10^{-3}	8.24×10^{-3}
13	8.76×10^{-3}	8.64×10^{-3}	8.60×10^{-3}	1.17×10^{-2}	7.82×10^{-3}	7.97×10^{-3}
14	8.53×10^{-3}	8.67×10^{-3}	8.65×10^{-3}	1.02×10^{-2}	7.77×10^{-3}	7.84×10^{-3}
15	8.50×10^{-3}	8.68×10^{-3}	8.67×10^{-3}	9.34×10^{-3}	7.74×10^{-3}	7.78×10^{-3}
16	8.53×10^{-3}	8.69×10^{-3}	8.68×10^{-3}	8.77×10^{-3}	7.73×10^{-3}	7.75×10^{-3}
17	8.57×10^{-3}	8.69×10^{-3}	8.69×10^{-3}	8.40×10^{-3}	7.72×10^{-3}	7.73×10^{-3}
18	8.60×10^{-3}	8.70×10^{-3}	8.69×10^{-3}	8.17×10^{-3}	7.72×10^{-3}	7.72×10^{-3}
19	8.63×10^{-3}	8.70×10^{-3}	8.70×10^{-3}	8.01×10^{-3}	7.72×10^{-3}	7.72×10^{-3}
20	8.65×10^{-3}	8.70×10^{-3}	8.70×10^{-3}	7.91×10^{-3}	7.72×10^{-3}	7.72×10^{-3}
21	8.67×10^{-3}	8.70×10^{-3}	8.70×10^{-3}	7.85×10^{-3}	7.72×10^{-3}	7.72×10^{-3}
22	8.68×10^{-3}	8.70×10^{-3}	8.70×10^{-3}	7.80×10^{-3}	7.72×10^{-3}	7.72×10^{-3}
23	8.68×10^{-3}	8.70×10^{-3}	8.70×10^{-3}	7.77×10^{-3}	7.72×10^{-3}	7.72×10^{-3}
24	8.69×10^{-3}	8.70×10^{-3}	8.70×10^{-3}	7.76×10^{-3}	7.72×10^{-3}	7.72×10^{-3}
25	8.69×10^{-3}	8.70×10^{-3}	8.70×10^{-3}	7.74×10^{-3}	7.72×10^{-3}	7.72×10^{-3}

Tab. 3: *Relative errors of different solution sequences.*

at Tab. 3, we notice, that Picard's method might not even have converged after 25 steps, therefore we distinguish the other methods as vastly superior. Interestingly, the step-size controlled solution-sequences perform better in ISMIP-B compared to ISMIP-A, contrary to Picard's method.

4.2.3 Vertical Resolution

Since we so far only comapred surface velocities, we visulized the diffrent layers in Fig. 16 and Fig. 17. As expected the velocity raises with each layer, reaching the maximum at the surface and following the basal topography.

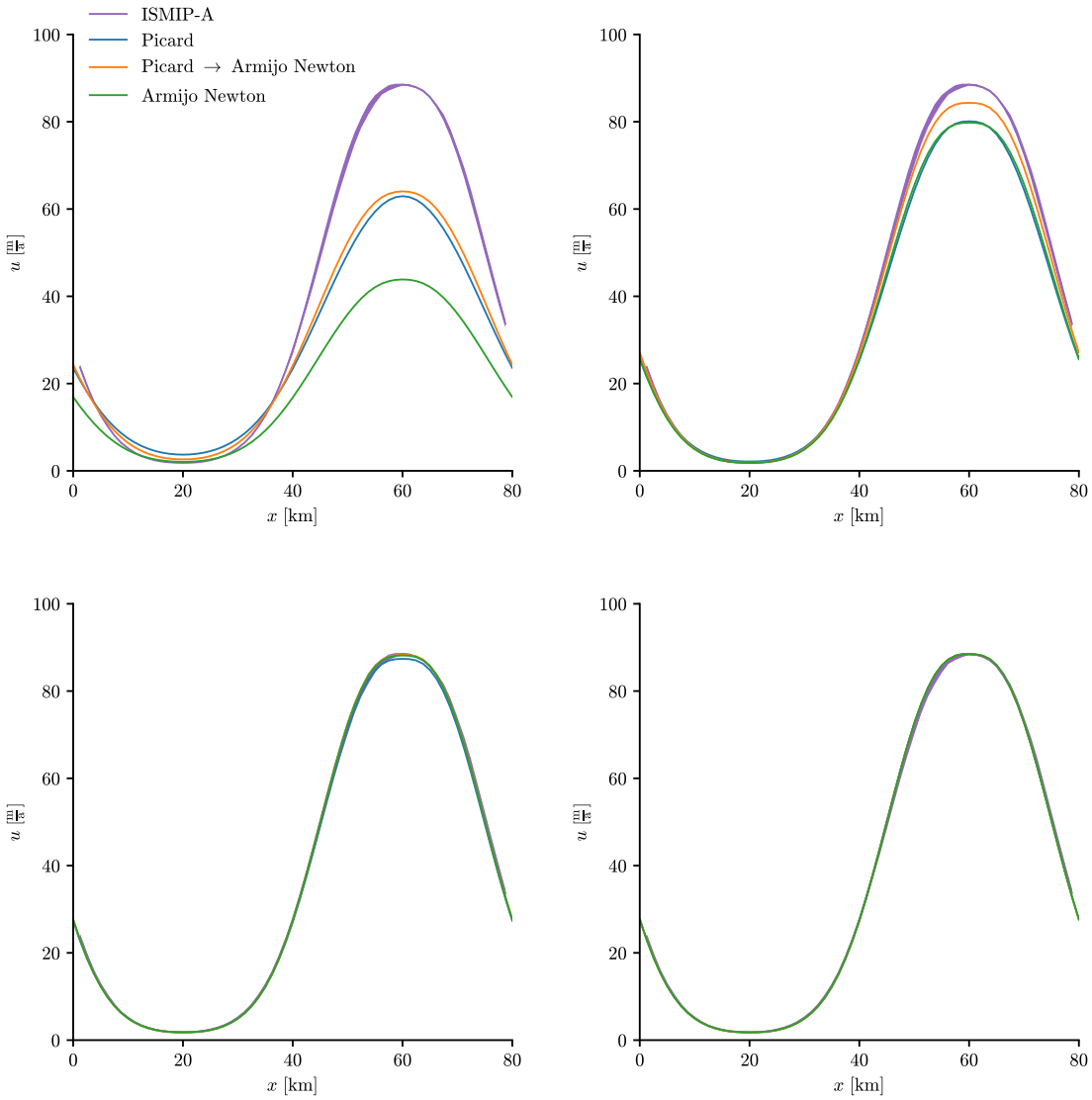

Fig. 12: *Prediction for ISMIP-A after 2 (top left), 5 (top right), 10 (bottom left), and 25 (bottom right) iterations.*

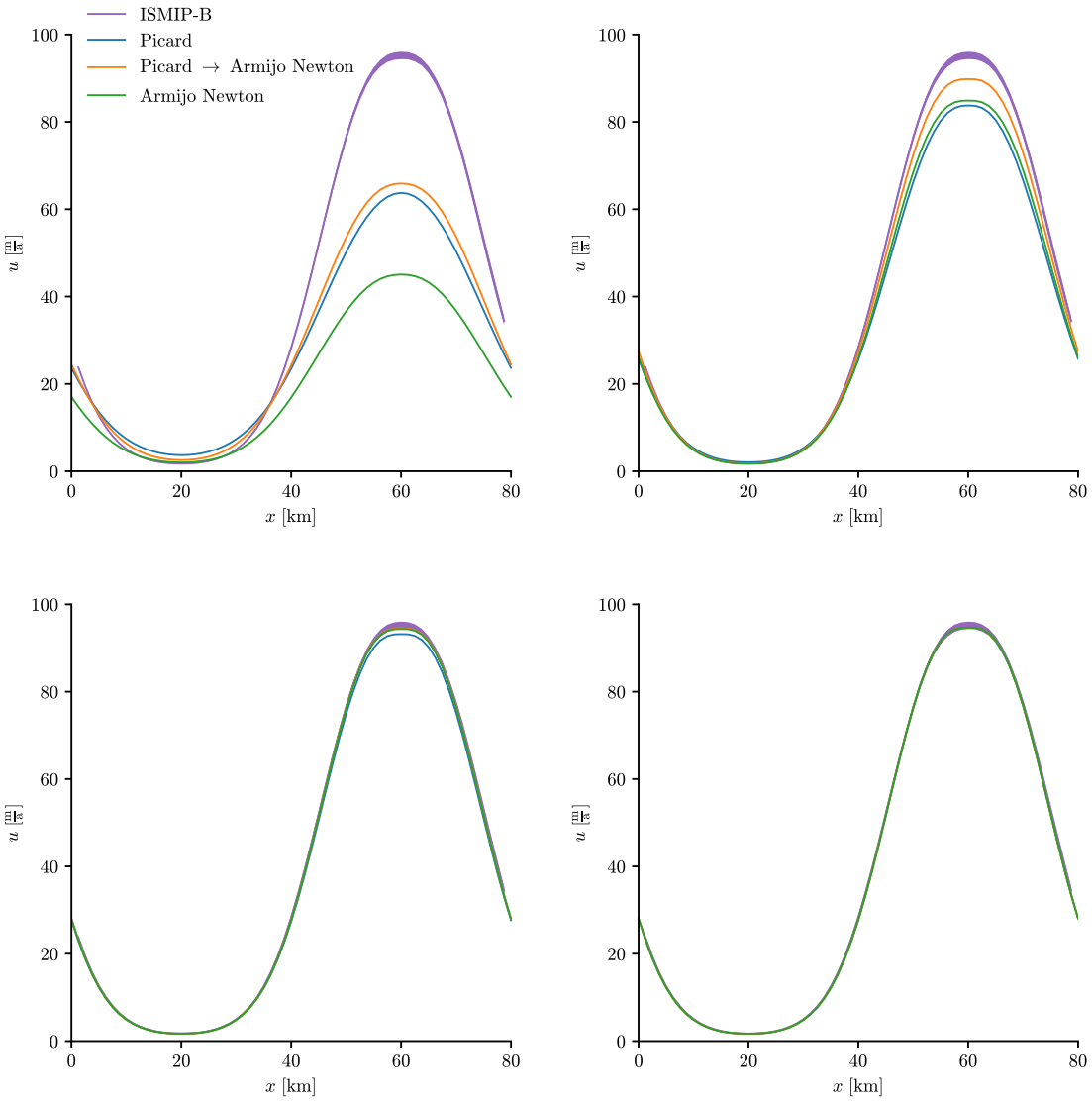

Fig. 13: *Prediction for ISMIP-A after 2 (top left), 5 (top right), 10 (bottom left), and 25 (bottom right) iterations.*

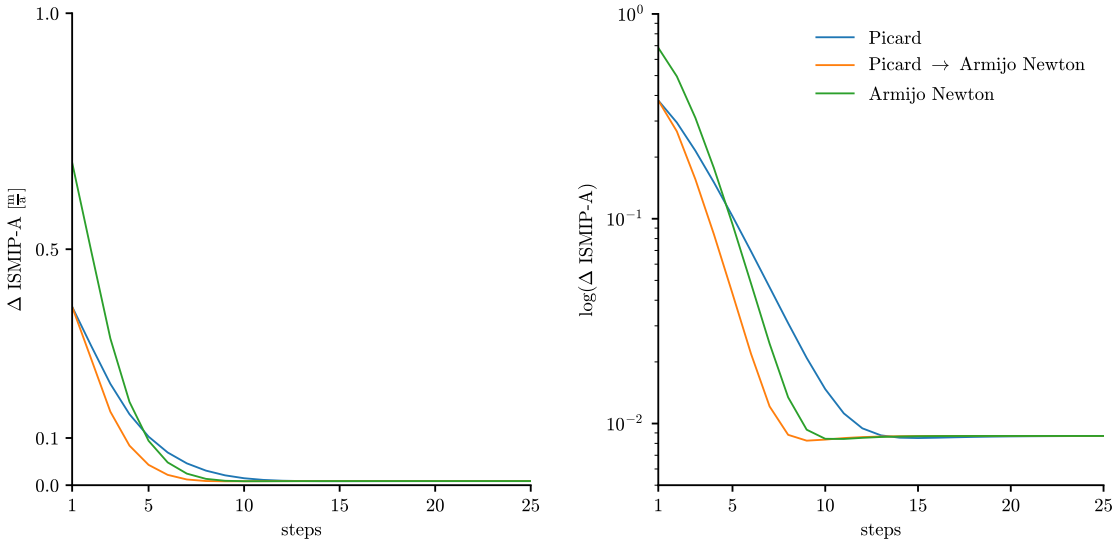

Fig. 14: *Relative errors of predictions for ISMIP-A.*

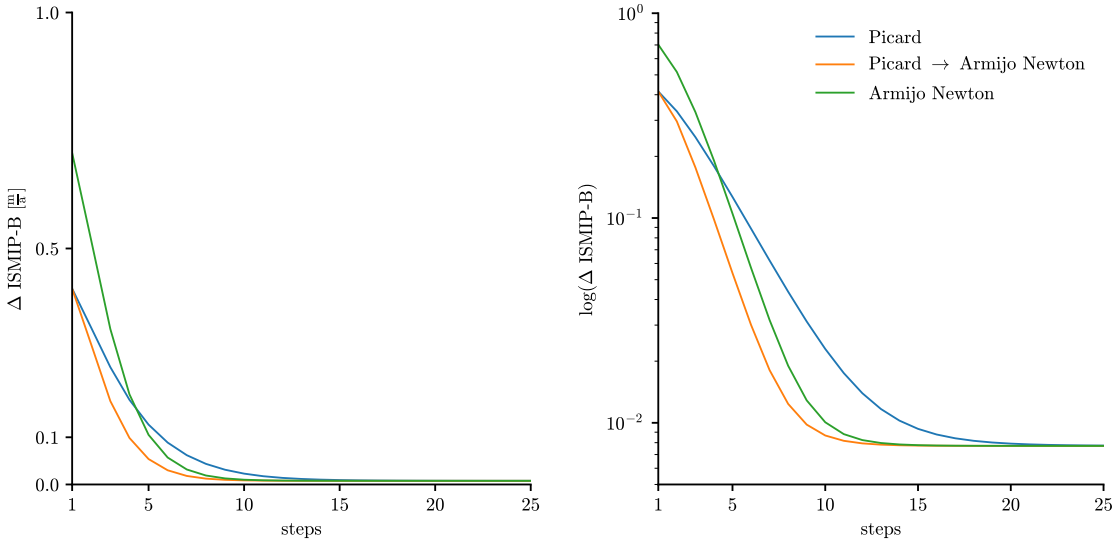

Fig. 15: *Relative errors of predictions for ISMIP-B.*

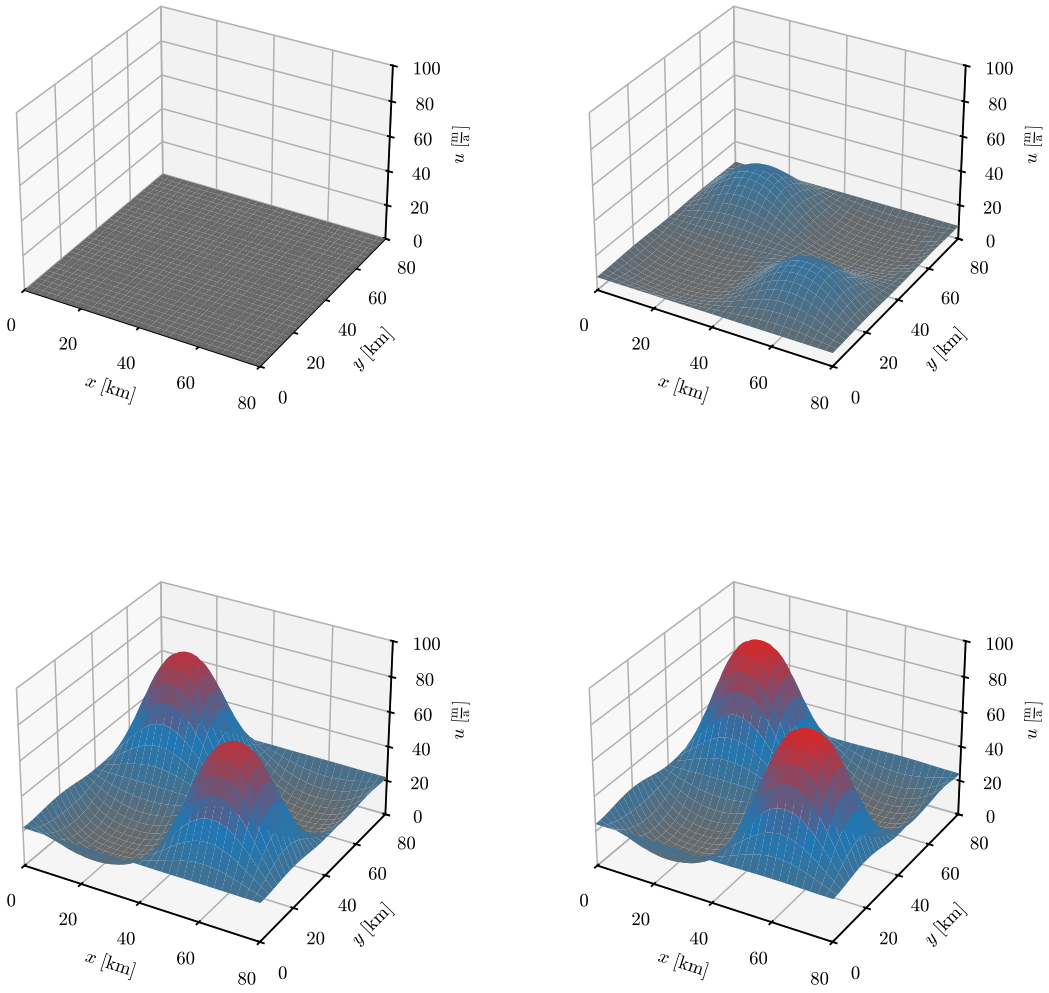

Fig. 16: *Horizontal velocity for experiment ISMPI-A at layers 1 (base, top left), 2 (top right), 6 (bottom left), and 12 (surface, bottom right). Red color indicates hight velocity.*

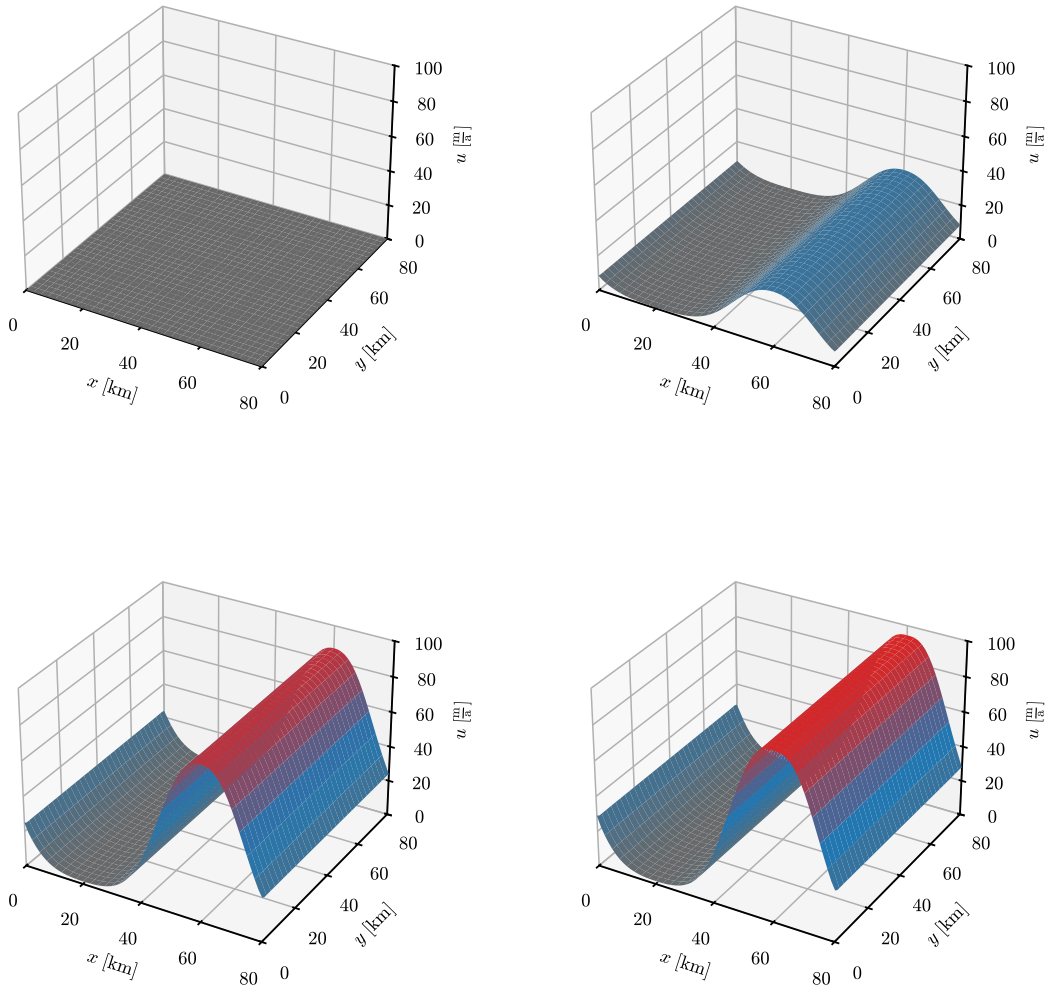

Fig. 17: *Horizontal velocity for experiment ISMPI-B at layers 1 (base, top left), 2 (top right), 6 (bottom left), and 12 (surface, bottom right). Red color indicates hight velocity.*

5 Discussion

After we evaluating our implementation of a CMake toolchain and Armijo-Newton method for ISSM, we discuss the observed results and try to answer our research question.

5.1 Software Engineering

While we observed, that ISSM does not strictly comply with the paradigm of FOSD, since we have several layers of extensive configurability, which are somewhat connected. The configuration space is not easily computed, compared to other frameworks, like DUNE. On the other hand, especially run-time features are encapsulated trough the Python API and, for example, individual source files for different solution sequences. Focused on our RQ1.1, we were able to infer several connected configuration at different layers of the program and visualize them in different FMs. Therefore, we can answer RQ1, that ISSM tries to provide a development environment, which allows and encourages the easy inclusion of new feature code. On the other hand, a lack of documentation, code comments and overall deeply nested code, makes it hard for an entry-level developer to get a hang on the structure of the source code.

5.2 Software Comprehension

It is evident, that even with a background in applied mathematics and computer science, common implementation practices, especially legacy code, needs a lot of reverse engineering. This problem becomes more evident, as the source code is sparsely documented, and the available user manual exclusively focuses on the API.

For example, we ran into initial difficulties understanding the sematic of Newton's method, which we were able to resolve, but which required understanding of the physical foundations.

While some implementation patterns are commonly used in similar circumstances, a software engineer might not see some side-effects immediately, and be confused on the intended behavior. Compared to other frameworks, like LLVM or DUNE, ISSM lacks the aspects of a modern C++ code-base hindering further development.

Regarding RQ2, and in particular RQ2.1, RQ2.2 and RQ2.3, we answer those questions through the described shortcomings of ISSM, resulting in various problems an new developer might encounter.

As example, looking at the implementation of automatic differentiation by Pleßke et al. [22], we noticed, that due to an user error all compile sources were included in the GIT-version-control-system. Since GIT tries to store the changes of each file, the resulting repository blew up to a size of over 16 gigabyte, possibly unknown to the developers.

With building CMAKE in a separate build folder, and excluding this folder from GIT, this oversight is no longer possible.

5.3 Improved Toolchain

The re-complication during development was an observed shortcoming of the old toolchain due to outdated and obscure design.

As answer to RQ3, we might have improved the user, and especially developer, experience with ISSM in a significant manner. In response RQ3.2, we have presented our faster SOTA toolchain in section 4.1. By analyzing the structure, providing a faster compilation toolchain, and setting the framework up for further improvements, we can reply to RQ3.1, that we assume, that the configuration is now way easier compared to the initial state of ISSM.

Note, that we enabled ISSM to be build with CLANG. This required resolving some wrong relative locations of header files, since CLANG seems less robust regarding missing definitions.

Building with NINJA, we never encountered any issue.

5.4 ISSM Python API

We did leave the PYTHON API mostly untouched. Plotting by default to a screen adapter instead of writing plots to a file is irritating, especially if we run ISSM on a virtual machine.

```
def parameterize(md, parametername):
    exec(compile(open(parametername).read(), parametername, 'exec'))
    return md
```

Fig. 18: *Exploitable PYTHON snippet.*

Nonetheless, we saw some ill-advised practices in the implementation of the API. We found the code snippet shown in Fig. 18, which implements “parametrization”

through executing a passed file from the file system of the user without any sanitization. As we work with large experiments, this file was possibly downloaded or passed on from other users making this highly deceptible to unintended or dangerous behavior. [9]

It is questionable, why the need for such a part of the parametrization arises, as the python front-end already encapsulated most of the functionality and the parametrization file itself also only contains commands, which could be just executed in place. A better approach, allowing the user to switch-out parts of the code with reading-in and rule-based verification of a configuration in text form, for example YAML, XML or JSON, as commonly encountered with other programs. Alternatively, a factory pattern might also suffice the need for further configurability.

5.5 Improving Newton' Method

For RQ4, we have shown section 4.2, that we can improve the naive implementing of Newton's method with step-size control. Furthermore, we can saw, that we converge faster compared to Picard's method. An initial guess seem less important, since the Newton-Armijo method converges already sufficiently fast. Therefore, we see no benefit in using this approach, and answer to RQ4.2, that both Newton-Armijo methods converge roughly after the same number of iterations. The quality of the simulation highly depends on the three-dimensional resolution of the problem space, in particular the number of mesh vertices and extrusion layers. We saw from initial test runs, that the simulation with less nodes converges to a prediction with a larger error compared to the reference of ISMIP. This behavior was already documented by Schmidt et al. [26], and we were able to reproduce similar results, answering our last research question RQ4.1.

5.6 Possibilities for further improvements

5.6.1 Further Improving Newton' Method

Not only does our approach results in a faster converging approximation, but open the possibilities for further improvements. For example, Schmidt et al. [26] have shown, that using a functional, instead of Armijo condition, as guidance for the condition, the calculate a exact step-size, allowing even faster convergence and especially possibly a better solution.

5.6.2 Automatic Differentiation

A huge disadvantage of using Newton's Method in general, is the additional overhead from the calculating the Jacobi matrix as derivative of the linear-equation system.

Instead of the calculation done by ISSM, this can be addressed by using other approaches, like automatic differentiation. Pleßke et al. [22] integrated automatic differentiation with ADOLc into ISSM for exactly this purpose, which would be negating the currently observed bottleneck.

Originally we intended to pursue this topic for this thesis in addition to step-size control. But, since we encountered issues with the build infrastructure, we prioritized reworking the compilation toolchain of ISSM. Fortunately, our results are a good starting point for a further investigation of novel approaches for the framework.

5.6.3 CMake

For the scope of this work we mainly focused on building a prototype toolchain including one of the most common configuration. Since ISSM is highly configurable, most options were neither tested nor further evaluated.

As we did keep the logic of the source file accumulation in the source folders, one would only have to include configuration options at the top level of CMake. This mostly includes telling the tool, where to look for new header and shared / static library archives. Moreover, this enables developers to easily include new external resources into the framework.

5.7 Threats to Validity

We tried to minimize the possibility of systematical errors through evaluating our approaches at different stages during development. As we base our work on the mathematical foundations from other authors, especially Schmidt et al. [26], our work comes down to faithfully adapting the proposed ideas. Fortunately, we can use ISMIP as reference for our simulation.

5.7.1 Implementation

Since the scope of this thesis does focus on implementing a numerical approach, we can only argue based on our understanding of the framework from reverse-engineering relevant parts.

There might be faster ways to archive similar results, which may be unknown to us. As we did not focus on deployment-ready code, we did implemented the proposed algorithms as a prototype.

5.7.2 Benchmarks

As already discussed, we had to adapt the implementation of the ISMIP benchmarks, provide by ISSM, since they used slightly different constants compared to the requirement of the benchmark. Further discrepancy might be undetected for other parts of the model. Since we were able to reproduce a very close match to the reference results, we can argue that the influence of such errors is small.

5.7.3 Scalability

Our results indicating an improvement of Newton's method are limited to the evaluated experiments and might differ for larger of more complex problems. On the plus side, since we looked into several combinations of mesh and number of extrusion layers for ISMIP-A and ISMIP-B, we can at least argue, that we observed explained variation of the simulation. A higher resolution mesh with more extrusion layers resulted in a closer fit to the reference.

Our novel approaches converged in fewer steps for all cases and might even be able to archive a closer fit, if evaluated on an even finer grid.

An natural extension would be, to evaluate our algorithm with a large scale real-world model, for example the Greenland ice-sheet, and investigate how Newton-Armijo performs performs in aspect to scalability.

6 Conclusion and Outlook

Numerical simulation are an important part of ongoing research. Climate simulation, changes to ice-sheets and rising sea level require extensive investigation.

With ISSM, we provide a framework to predict ice-flow and the behavior modeled through complex systems.

First, we focused on improving the user experience with a faster toolchain. To guide configuration of the model, we investigated the paradigms of FOSD and found that ISSM complies with a feature-oriented development pipeline.

Through using CMake with Ninja and Clang vastly reduces the compilation overhead.

In addition, we introduced a new solution sequence, by incorporating the Armijo condition with Newton's method, outperforming the default algorithm.

Controlling the step size during iteration seem like a good starting point for further improvements of ISSM.

References

[1] Clang: A C Language Family Frontend for LLVM, https://clang.llvm.org/, last accessed 25 Jan 2026

[2] DUNE Numerics, https://www.dune-project.org/, last accessed 25 Jan 2026

[3] GNU Operating System, https://www.gnu.org/, last accessed 25 Jan 2026

[4] ISSM, https://science.jpl.nasa.gov/projects/issm/, last accessed 25 Jan 2026

[5] Ninja, https://ninja-build.org/, last accessed 25 Jan 2026

[6] PETSc, https://petsc.org/release/, last accessed 25 Jan 2026

[7] The DUNE Cheat Sheet, https://www.dune-project.org/pdf/dune-cheat-sheet.pdf, last accessed 25 Jan 2026

[8] The LLVM Compiler Infrastructure, https://llvm.org/, last accessed 25 Jan 2026

[9] Exploits of a Mom (2024), https://xkcd.com/327/, last accessed 25 Jan 2026

[10] Aho, A.V., Sethi, R., Ullman, J.D.: Compilers: Principles, Techniques and Tools. Addison-Wesley (1988)

[11] Apel, S., Kästner, C.: An Overview of Feature-oriented Software Development (2009). https://doi.org/10.5381/jot.2009.8.5.c5

[12] Argonne National Laboratory: PETSc Users Manual (2016)

[13] Dennis, J.E., Schnabel, R.B.: Numerical Methods for Unconstrained Optimization and Nonlinear Equations. Society for Industrial and Applied Mathematics (1996). https://doi.org/10.1137/1.9781611971200

[14] International Organization for Standardization: ISO/IEC 14882:2024(E) - Programming Language C++ (2023)

[15] Kitware Inc.: CMake, https://cmake.org/, last accessed 25 Jan 2026

[16] Lattner, C., Adve, V.: LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation. In: International Symposium on Code Generation and Optimization. Palo Alto, California, USA (2004)

[17] Logan, J.D.: A First Course in Differential Equations. Springer, 2. edn. (2011)

[18] MacKenzie, D., Elliston, B., Demaille, A.: Autoconf. Free Software Foundation Inc. (2023)

[19] MacKenzie, D., Tromey, T., Duret-Lutz, A., Wildenhues, R., Lattarini, S.: Automake. Free Software Foundation Inc. (2025)

[20] Morlighem, M., et al.: Ice-sheet and Sea-level System Model 2025 (4.24) User Guide (2025)

[21] Pattyn, F., Perichon, L., Aschwanden, A., et al.: Benchmark Experiments for Higher-order and Full-Stokes Ice Sheet Models (ISMIP-HOM) (2008)

[22] Pleßke, S.M.: Development and Performance Analysis of a Jacobian-free Newton-Krylov Method Using Automatic Differentiation Within the Ice Sheet and Sea Level System Model (ISSM) (2009). https://doi.org/10.5381/jot.2009.8.5.c5

[23] Pollock, S., Rebholz, L., Tu, X., Xiao, M.: Analysis of the Picard-Newton Iteration for the Navier-Stokes Equations: Global Stability and Quadratic Convergence (2024). https://doi.org/10.48550/ARXIV.2402.12304

[24] Sander, O.: DUNE - The Distributed and Unified Numerics Environment. Springer (2020)

[25] Sattler, F.: A Variability-aware Feature-Region Analyzer in LLVM. Master's thesis, University of Passau, Germany (2017)

[26] Schmidt, N., Humbert, A., Slawig, T.: Assessing the benefits of approximately exact step sizes for picard and newton solver in simulating ice flow (fenics-full-stokes) **17**(12) (2024). https://doi.org/10.5194/gmd-17-4943-2024

[27] Schubert, P.D., Hermann, B., Bodden, E.: PhASAR: An Inter-procedural Static Analysis Framework for C/C++. In: Tools and Algorithms for the Construction and Analysis of Systems. Lecture Notes in Computer Science, Springer (2019)

[28] Stroustrup, B.: The C++ Programming Language. Addison-Wesley, 4. edn. (2013)

[29] Thunberg, G.: The Climate Book. Penguin Random House (2022)

[30] Zehra, F., Javed, M., Khan, D., Pasha, M.: Comparative Analysis of C++ and Python in Terms of Memory and Time (2020). https://doi.org/10.20944/preprints202012.0516.v1

Source code available on GitHub.

Figures

Tables

Appendix

```
uf = old_uf->Duplicate();
old_uf->Copy(uf);

pJf = pf->Duplicate();
Kff->MatMult(uf, pJf);
pJf->AXPY(pf, -1.0);
pJf->Scale(-1.0);

CreateJacobianMatrixx(&Jff, femmodel, kmax);

Solverx(&duf, Jff, pJf, nullptr, nullptr, femmodel->parameters);

IssmDouble alpha = 1;
IssmDouble gamma = 1e-10;

IssmDouble gradient = pJf->Norm(NORM_TWO);

old_uf->Copy(uf);
uf->AXPY(duf, alpha);
Kff->MatMult(uf, pJf);

IssmDouble min_term_old = pJf->Norm(NORM_TWO);
IssmDouble min_term_new;

for (;;) {
    old_uf->Copy(uf);
    uf->AXPY(duf, alpha);
    Kff->MatMult(uf, pJf);

    min_term_new = pJf->Norm(NORM_TWO);

    if (min_term_new <= min_term_old - gamma * alpha * gradient) {
        break;
    }

    alpha = .5 * alpha;
}
```

App. I: *Implementation of Armijo step-size control.*

```
import numpy as np

ISSM_DIR = Path(...).absolute()

sys.path.append(str(ISSM_DIR / 'bin'))
sys.path.append(str(ISSM_DIR / 'lib'))
sys.path.append(str(ISSM_DIR / 'share'))
sys.path.append(str(ISSM_DIR / 'share/proj'))

from issmversion import issmversion

from model import *
from squaremesh import squaremesh
from plotmodel import plotmodel
from export_netCDF import export_netCDF
from loadmodel import loadmodel
from setmask import setmask
from parameterize import parameterize
from setflowequation import setflowequation
from socket import gethostname
from solve import solve

md = model()

L = 80_000
md = squaremesh(md, L, L, 65, 65)
md = setmask(md, '', '')

md.miscellaneous.name = 'IsmipA_cor'

md.geometry.surface = -md.mesh.x * np.tan(0.5 * np.pi / 180.0)
L = np.nanmax(md.mesh.x) - np.nanmin(md.mesh.x)
md.geometry.base = md.geometry.surface - 1000.0 + 500.0
    * np.sin(md.mesh.x * 2.0 * np.pi / L)
    * np.sin(md.mesh.y * 2.0 * np.pi / L)
md.geometry.thickness = md.geometry.surface - md.geometry.base

md.friction.coefficient = 200.0 * np.ones((md.mesh.numberofvertices))
md.friction.p = np.ones(md.mesh.numberofelements)
md.friction.q = np.ones(md.mesh.numberofelements)

md.geometry.base = md.geometry.surface - 1000.0 + 500.0
    * np.sin(md.mesh.x * 2.0 * np.pi / L)

md.materials.rho_ice = 910
md.materials.rheology_B = 6.808188376e7
    * np.ones(md.mesh.numberofvertices)
md.materials.rheology_n = 3 * np.ones(md.mesh.numberofelements)

md = SetIceSheetBC(md)

md = md.extrude(12, 1)
md = setflowequation(md, 'HO', 'all')

md.stressbalance.spcvx = np.nan * np.ones(md.mesh.numberofvertices)
md.stressbalance.spcvy = np.nan * np.ones(md.mesh.numberofvertices)

basalnodes = np.nonzero(md.mesh.vertexonbase)
md.stressbalance.spcvx[basalnodes] = 0.0
md.stressbalance.spcvy[basalnodes] = 0.0

md.stressbalance.vertex_pairing = ...

md.cluster = generic('name', gethostname(), 'np', 2)

md.stressbalance.isnewton = 3
md.stressbalance.maxiter = ...

md = solve(md, 'Stressbalance')

export_netCDF(md, "ISMIP-StressBalance.nc")
```

App. II: *Implementation of ISMIP-A.*